

Project work, part 4 - Machine Learning -1

Orie Kimura

November 28, 2025

Deployment Challenges

My connection to remote MongoDB Atlas has become extremely slow recently.

- Although I've successfully uploaded data to remote MongoDB Atlas previously (as shown by the execution log), I cannot access/use it within the Streamlit Cloud application.
- I was forced to finish development of the Streamlit app using a locally installed MongoDB instance. Consequently, I haven't been able to test the Streamlit Cloud application, despite having pushed all files to GitHub. Everything works correctly on my PC, however.

General

Links to public GitHub repository and Streamlitapp for the compulsory work.

<https://github.com/oriekimura123/IND320-Projectwork>

<https://oriekimura123-ind320-project.streamlit.app/>

Tasks

Jupyter Notebook

- Use the Elhub API to retrieve hourly production data for all price areas using PRODUCTION_PER_GROUP_MBA_HOUR for all days and hours of the years 2022 - 2024.
 - Handle these data the same way as in part 2 of the project, appending the new data after the 2021 data, both in Cassandra (using Spark - see updated advice in the installation page if you struggle) and MongoDB.
 - Using new tables in your databases, but otherwise the same strategy, retrieve hourly consumption data for all price areas using CONSUMPTION_PER_GROUP_MBA_HOUR for all days and hours of the years 2021 -2024.

I created functions to handle both production and consumption data:

fetch_elhub_data, setup_spark_session, create_cassandra_keyspace_and_table and

write_to_cassandra

I also had to create a new table/data structure to accommodate the new functions. Consequently, I dropped the old production data tables in Cassandra and MongoDB, and created new tables.

```
In [1]: # SETUP, IMPORTS, AND CONFIGURATION

# --- Environment Setup ---
import os
from typing import List, Dict
# from datetime import date, time
# from urllib.parse import quote

# Paths configuration
SPARK_HOME = "C:\\Spark\\spark-3.5.1-bin-hadoop3"
HADOOP_HOME = "C:\\Hadoop\\hadoop-3.3.1"
JAVA_HOME = "C:\\Program Files\\Microsoft\\jdk-21.0.8.9-hotspot"

# Set environment variables
os.environ.update({
    "SPARK_HOME": SPARK_HOME,
    "HADOOP_HOME": HADOOP_HOME,
    "JAVA_HOME": JAVA_HOME,
    "PATH": os.environ["PATH"] + os.pathsep + os.path.join(SPARK_HOME, "bin"),
    "PYSPARK_PYTHON": "python",
    "PYSPARK_DRIVER_PYTHON": "python",
    "PYSPARK_HADOOP_VERSION": "without",
    "SPARK_CONF_DIR": os.path.join(SPARK_HOME, "conf")
})

# Database configuration
CASSANDRA_KEYSPACE = "my_keyspace"
# Use descriptive table names for the full dataset (2021-2024)
CASSANDRA_TABLE_PRODUCTION_FULL = "production_data_2021_2024"
CASSANDRA_TABLE_CONSUMPTION_FULL = "consumption_data_2021_2024"
MONGO_DATABASE = "elhub_data"
MONGO_COLLECTION_PRODUCTION_FULL = "production_data_2021_2024"
MONGO_COLLECTION_CONSUMPTION_FULL = "consumption_data_2021_2024"

# API Configuration
BASE_URL = "https://api.elhub.no/energy-data/v0/price-areas"
DATASET_PRODUCTION = "dataset=PRODUCTION_PER_GROUP_MBA_HOUR"
DATASET_CONSUMPTION = "dataset=CONSUMPTION_PER_GROUP_MBA_HOUR"
PRICE_AREAS: List[str] = ["N01", "N02", "N03", "N04", "N05"]

# Time constants
MAX_RETRIES = 3
DELAY_SECONDS = 0.5

# Define the test range (using a small range is safer and faster for testing)
from datetime import date
START_date = date(2021, 1, 1)
END_date = date(2024, 12, 31)
```

```
print(f"Data range set to: {START_date} to {END_date}")
```

Data range set to: 2021-01-01 to 2024-12-31

```
In [2]: # Fetch elhub data
        # only pricearea, datatype, groupname, starttime, quantitykwh are fetced

        from utils.data_loaders import fetch_elhub_data

        data = fetch_elhub_data(
            START_date,
            END_date,
            DATASET_CONSUMPTION,
            PRICE_AREAS,
            BASE_URL,
            MAX_RETRIES,
            DELAY_SECONDS
        )
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```

Fetching data from :https://api.elhub.no/energy-data/v0/price-areas/N05?dataset=CONSUMPTION_PER_GROUP_MBA_HOUR&startDate=2024-11-01T00%3A00%3A00%2B01%3A00&endDate=2024-12-01T00%3A00%3A00%2B01%3A00
<- Successfully retrieved 1 records for N05.
Fetching data from :https://api.elhub.no/energy-data/v0/price-areas/N05?dataset=CONSUMPTION_PER_GROUP_MBA_HOUR&startDate=2024-12-01T00%3A00%3A00%2B01%3A00&endDate=2025-01-01T00%3A00%3A00%2B01%3A00
<- Successfully retrieved 1 records for N05.
Total_records:0

```

In [3]: *# Convert to Pandas for quick verification*

```

import pandas as pd

df_pandas = pd.DataFrame(data)

if not df_pandas.empty:
    print("Data Head:")
    print(df_pandas.head())
    print("\nColumns Check:")
    print(df_pandas.columns)
else:
    print("ERROR: data list is empty.")

```

Data Head:

	pricearea	datatype	groupname	starttime	quantitykwh
0	N01	Consumption	cabin	2021-01-01T00:00:00+01:00	177071.56
1	N01	Consumption	cabin	2021-01-01T01:00:00+01:00	171335.12
2	N01	Consumption	cabin	2021-01-01T02:00:00+01:00	164912.02
3	N01	Consumption	cabin	2021-01-01T03:00:00+01:00	160265.77
4	N01	Consumption	cabin	2021-01-01T04:00:00+01:00	159828.69

Columns Check:

```

Index(['pricearea', 'datatype', 'groupname', 'starttime', 'quantitykwh'], dtype='object')

```

In [4]: *# Setup Spark Session*

```

from utils.database_interaction import setup_spark_session
spark = setup_spark_session("CassandraToMongo")
print("\nSpark Session established.")

```

Spark Session established.

In [5]: *# Create the Spark DataFrame from the list*

```

from pyspark.sql.types import StructType, StructField, StringType, DoubleType
from pyspark.sql.functions import monotonically_increasing_id
from pyspark.sql.types import LongType

schema = StructType([
    StructField("pricearea", StringType(), False),
    StructField("datatype", StringType(), False),
    StructField("groupname", StringType(), True),
    StructField("starttime", StringType(), False),
    StructField("quantitykwh", DoubleType(), False)
])

# Create the Spark DataFrame from the list of dicts
df_spark = spark.createDataFrame(data, schema)

```

```

# Add a unique integer ID column named 'ind' to the DataFrame
# ind should be LongType and the primary key in Cassandra

df_spark = df_spark.withColumn(
    "ind",
    monotonically_increasing_id().cast(LongType())
)

df_spark.printSchema() # Verify the schema is correct

```

```

root
|-- pricearea: string (nullable = false)
|-- datatype: string (nullable = false)
|-- groupname: string (nullable = true)
|-- starttime: string (nullable = false)
|-- quantitykwh: double (nullable = false)
|-- ind: long (nullable = false)

```

```

In [6]: # Cassandra
# Setup
# IMPORTANT: Ensure your Docker container is running and wait ~90 seconds
# before running this cell to allow Cassandra to fully start.
from utils.database_interaction import create_cassandra_keyspace_and_table
from cassandra.cluster import Cluster

cluster = Cluster(['127.0.0.1'])
session = cluster.connect()

create_cassandra_keyspace_and_table(
    session,
    CASSANDRA_KEYSPACE,
    CASSANDRA_TABLE_CONSUMPTION_FULL # Use the new table name
)

from utils.database_interaction import write_to_cassandra

# Write data to Cassandra
write_to_cassandra(
    df_spark.write,
    CASSANDRA_KEYSPACE,
    CASSANDRA_TABLE_CONSUMPTION_FULL
)
print(f"Data written to {CASSANDRA_TABLE_CONSUMPTION_FULL}.")

```

Table 'my_keyspace.consumption_data_2021_2024' dropped (if it existed).
 Keyspace 'my_keyspace' confirmed/created.
 Session set to keyspace 'my_keyspace'.
 Table 'consumption_data_2021_2024' confirmed/created.
 Data written to consumption_data_2021_2024.

```

In [7]: # Verify data was written to Cassandra by reading it back
# Read the data from Cassandra into a DataFrame (df_read)
df_read = spark.read \
    .format("org.apache.spark.sql.cassandra") \
    .options(table=CASSANDRA_TABLE_CONSUMPTION_FULL, keyspace=CASSANDRA_KEYSPACE) \

```

```

        .load()

# Show the schema to confirm types were read correctly
print("Schema after reading from Cassandra:")
df_read.printSchema()

# Show the first 5 rows of data
print("\nFirst 5 rows read from Cassandra:")
df_read.show(5)

# Use the .count() action
row_count = df_read.count()
print(f"Total number of rows in the table: {row_count}")

```

Schema after reading from Cassandra:

```

root
|-- ind: long (nullable = false)
|-- datatype: string (nullable = true)
|-- groupname: string (nullable = true)
|-- pricearea: string (nullable = true)
|-- quantitykwh: double (nullable = true)
|-- starttime: string (nullable = true)

```

First 5 rows read from Cassandra:

```

+-----+-----+-----+-----+-----+-----+
|      ind|  datatype|groupname|pricearea|quantitykwh|      starttime|
+-----+-----+-----+-----+-----+-----+
|77309481212|Consumption|  cabin|    N05|   17490.512|2021-08-25T01:00:...|
|85899370590|Consumption| tertiary|    N05|    271168.4|2022-03-29T03:00:...|
|25769870797|Consumption| primary|    N02|    45139.297|2023-07-02T10:00:...|
|68719524329|Consumption|  cabin|    N04|    18357.34|2023-06-26T22:00:...|
|77309424088|Consumption| primary|    N04|    72128.305|2024-04-19T21:00:...|
+-----+-----+-----+-----+-----+-----+

```

only showing top 5 rows

Total number of rows in the table: 876600

```

In [8]: # Mongo DB
# Setup
import streamlit as st
try:
    MONGO_URI = st.secrets["mongo"]["uri"]
except KeyError:
    st.error("MongoDB URI not found in Streamlit secrets. Check your .streamlit/sec
    st.stop()

# insert the data into MongoDB
df_spark.write \
    .format("mongodb") \
    .mode("overwrite") \
    .option("database", MONGO_DATABASE) \
    .option("collection", MONGO_COLLECTION_CONSUMPTION_FULL) \
    .save()

```

```
In [9]: # Verify data was written to MongoDB by reading it back
try:
    df_mongo_data = (
        spark.read
        .format("mongodb")
        # specify the database and collection here, as the connection URI is set gl
        .option("database", MONGO_DATABASE)
        .option("collection", MONGO_COLLECTION_CONSUMPTION_FULL)
        .load()
    )

    print("\n--- First 5 Rows of Data ---")
    df_mongo_data.show(5)

except Exception as e:
    print(f"\n An error occurred during the read operation.")
    print(f"Please double-check your database/collection names and network access:")

finally:
    # Stop the Spark session when finished
    spark.stop()
```

```
--- First 5 Rows of Data ---
+-----+-----+-----+-----+-----+-----+-----+
+-----+
|_id|datatype|groupname|ind|pricearea|quantitykwh|
starttime|
+-----+-----+-----+-----+-----+-----+-----+
+-----+
|6919b408ca2c42f76...|Consumption|cabin|25769803776|N02|120631.28|2022-0
1-01T16:00:...|
|6919b408ca2c42f76...|Consumption|cabin|25769803777|N02|120932.58|2022-0
1-01T17:00:...|
|6919b408ca2c42f76...|Consumption|cabin|25769803778|N02|119658.36|2022-0
1-01T18:00:...|
|6919b408ca2c42f76...|Consumption|cabin|25769803779|N02|115860.55|2022-0
1-01T19:00:...|
|6919b408ca2c42f76...|Consumption|cabin|25769803780|N02|110932.484|2022-0
1-01T20:00:...|
+-----+-----+-----+-----+-----+-----+-----+
+-----+
only showing top 5 rows
```

```
In [10]: spark.stop()
session.shutdown()
cluster.shutdown()
print("\nSpark and Cassandra connections closed.")
```

Spark and Cassandra connections closed.

to be continued to part 2 of .ipynb file...